

Classic Embeddings: LSA & LDA

Lecture 7: The Foundations of Distributional Semantics

PSYC 51.07: Models of Language and Communication - Week 3
Winter 2026

Today's Lecture

1.  **The Distributional Hypothesis**
2.  **Vector Space Models**
3.  **Latent Semantic Analysis (LSA)**
4.  **Latent Dirichlet Allocation (LDA)**
5.  **Modern Topic Modeling: BERTopic**
6.  **Evaluation & Applications**

Goal: Understand how classic methods represent meaning through co-occurrence

The Fundamental Question 🤔

How do we represent the meaning of words in a way that computers can understand?

How do we represent the meaning of words in a way that computers can understand?

Traditional Approach:

- Dictionaries
- Taxonomies (WordNet)
- Manual feature engineering
- Knowledge bases

Limitation: Labor-intensive, hard to scale

Distributional Approach:

- Learn from data
- Context-based
- Scalable
- Unsupervised

Key Idea: Let the data tell us what words mean!

“You shall know a word by the company it keeps”

— J.R. Firth (1957)

"You shall know a word by the company it keeps"

— J.R. Firth (1957)

Key Idea: Words that appear in similar contexts tend to have similar meanings.

Example

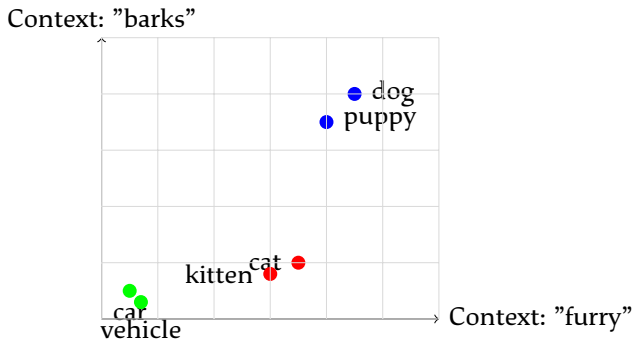
- "The **cat** sat on the mat"
- "The **dog** sat on the mat"
- "The **kitten** sat on the mat"

→ *cat, dog, kitten* are semantically related

Reference: Firth, J.R. (1957). "A synopsis of linguistic theory 1930-1955"

From Words to Vectors

Goal: Represent each word as a point in high-dimensional space



Problem: Real vocabularies have 10,000+ words, contexts are even more numerous!

→ We need **dimensionality reduction** 

Building a Term-Document Matrix

Step 1: Count how often each word appears in each document

Term	Doc1	Doc2	Doc3	Doc4
dog	3	0	2	0
cat	0	4	1	0
computer	0	0	0	5
code	0	0	0	3
animal	2	3	2	0

Issues:

- Very sparse (mostly zeros)
- High dimensional (vocab size $\times$ doc count)
- Doesn't capture semantic relationships

Building a Term-Document Matrix

Step 1: Count how often each word appears in each document

Term	Doc1	Doc2	Doc3	Doc4
dog	3	0	2	0
cat	0	4	1	0
computer	0	0	0	5
code	0	0	0	3
animal	2	3	2	0

Issues:

- Very sparse (mostly zeros)
- High dimensional (vocab size $\times$ doc count)
- Doesn't capture semantic relationships

Solution: Apply TF-IDF weighting and dimensionality reduction!

Term Frequency - Inverse Document Frequency

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Term Frequency (TF):

$$\text{TF}(t, d) = \frac{\text{count}(t, d)}{\sum_{t'} \text{count}(t', d)}$$

How often does term t appear in document d ?

Rewards frequent terms in document

Inverse Document Frequency (IDF):

$$\text{IDF}(t) = \log \frac{N}{|\{d : t \in d\}|}$$

How rare is term t across all documents?

Penalizes common words like "the", "and"

Latent Semantic Analysis (LSA)

The OG of semantic embeddings (1990)

Algorithm:

1. Build term-document matrix X
2. Apply TF-IDF weighting
3. Perform SVD: $X = U\Sigma V^T$
4. Keep top k dimensions
5. Use U_k as word embeddings

$$\begin{array}{c} \text{Terms} \\ \text{Docs} \end{array} \begin{array}{|c|} \hline X \\ \hline \end{array} = \begin{array}{|c|} \hline U \\ \hline \end{array} \begin{array}{|c|} \hline \Sigma \\ \hline \end{array} \begin{array}{|c|} \hline V^T \\ \hline \end{array}$$

Key Innovation:

- Discovers latent topics
- Solves synonymy problem
- Matrix factorization
- Captures semantic similarity

Matrix Interpretation:

- U : word-topic associations
- Σ : topic strengths
- V^T : topic-document associations

Reference: Deerwester et al. (1990) - "Indexing by Latent Semantic Analysis"

Example: Discovering latent topics

Original Dimensions:

- 50,000 terms
- 10,000 documents
- Matrix: $50k \times 10k$
- Sparse and noisy

After LSA:

- 100-300 latent dimensions
- Dense representation
- Captures semantic patterns
- Easier to compute

Discovered Topics (example):

Topic 1: Technology

computer (0.8), software (0.7), code (0.6), algorithm (0.5)

Topic 2: Animals

dog (0.9), cat (0.8), pet (0.7), animal (0.6)

Topic 3: Sports

game (0.8), team (0.7), player (0.6), win (0.5)

Words with similar topic distributions are semantically similar!

LSA Limitations

Computational Issues:

- SVD is expensive: $O(\min(nm^2, n^2m))$
- Hard to update with new documents
- Memory intensive for large corpora

Modeling Issues:

- Linear relationships only
- Bag-of-words (loses word order)
- No polysemy handling
- Negative values (hard to interpret)

Example Problem: Polysemy

Word: "bank"

1. Financial institution
2. River bank
3. Banking (airplane maneuver)

LSA gives **one vector** for all meanings!

But...

- Still useful for many tasks
- Foundation for modern methods
- Fast for medium-sized corpora

Latent Dirichlet Allocation (LDA)

A probabilistic approach to topic modeling

What if we model documents as probabilistic mixtures of topics?

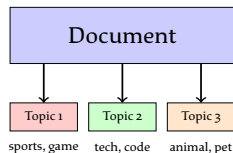
Key Assumptions:

1. Each document is a mixture of topics
2. Each topic is a distribution over words
3. The mixture proportions vary per document

Advantages over LSA:

- Probabilistic interpretation
- Non-negative weights
- Better for topic discovery
- More interpretable

Reference: Blei et al. (2003) - "Latent Dirichlet Allocation"



Mix: 30%, 60%, 10%

How LDA imagines documents are created:

Generative Process:

1. Choose number of topics K
2. For each topic k :
 - Draw word distribution $\phi_k \sim \text{Dir}(\beta)$
3. For each document d :
 - Draw topic distribution $\theta_d \sim \text{Dir}(\alpha)$
 - For each word position n :
 - Choose topic $z_{dn} \sim \text{Mult}(\theta_d)$
 - Choose word $w_{dn} \sim \text{Mult}(\phi_{z_{dn}})$

Key Parameters:

- α : Document-topic density
 - Low $\alpha \rightarrow$ few topics per doc
 - High $\alpha \rightarrow$ many topics per doc
- β : Topic-word density
 - Low $\beta \rightarrow$ focused topics
 - High $\beta \rightarrow$ general topics

Inference:

- Given documents, infer topics
- Use Gibbs sampling or variational inference
- Iterative process

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

# Prepare documents
documents = [
    "The cat sat on the mat",
    "The dog ran in the park",
    "Machine learning is amazing",
    # ... more documents
]

# Create bag-of-words representation
vectorizer = CountVectorizer(max_features=1000, stop_words='
    english')
doc_term_matrix = vectorizer.fit_transform(documents)
vocab = vectorizer.get_feature_names_out()
```

LDA Example: Topic Discovery

Example Topics from News Corpus:

Topic 1: Politics (20%)

government, election, president, vote, policy, senate, congress, party

Topic 2: Sports (15%)

game, team, player, score, win, season, coach, championship

Topic 3: Technology (25%)

software, computer, data, algorithm, system, internet, code, AI

Topic 4: Finance (40%)

market, stock, price, investment, economy, trade, bank, profit

Combining neural embeddings with topic models

BERTopic Pipeline:

1. Embed documents with BERT
2. Reduce dimensionality with UMAP
3. Cluster with HDBSCAN
4. Generate topics via c-TF-IDF
5. Fine-tune with
MaximalMarginalRelevance

Advantages:

- Leverages pre-trained models
- Better semantic understanding
- Automatic topic number detection
- Dynamic topic modeling

```
from bertopic import BERTopic
from sentence_transformers
    import SentenceTransformer

# Initialize
embedding_model =
    SentenceTransformer(
        "all-MiniLM-L6-v2"
    )

model = BERTopic(
    embedding_model=
        embedding_model,
    language="english",
    calculate_probabilities=
```

LSA vs. LDA vs. BERTopic

Property	LSA	LDA	BERTopic
Approach	Matrix factorization	Probabilistic	Neural + Clustering
Interpretability	Medium	High	High
Topic coherence	Medium	High	Very High
Computation	Fast	Medium	Slow
Scalability	Good	Good	Medium
Pre-training	No	No	Yes
Hyperparameters	Few	Many	Many
Topic number	Fixed	Fixed	Automatic

Recommendations

- **LSA:** Quick exploration, large-scale retrieval
- **LDA:** Interpretable topics, when you know topic count

• **BERTopic:** State-of-the-art quality exploratory analysis

Evaluating Topic Models

Intrinsic Metrics:

- **Perplexity:** Lower is better
 - Measures likelihood
 - Can be misleading!
- **Topic Coherence:** Higher is better
 - Measures semantic similarity
 - Better correlation with human judgment
 - CV, UCI, UMass variants
- **Topic Diversity:**
 - Unique words across topics
 - Avoids redundant topics

Extrinsic Metrics:

- Document classification accuracy
- Information retrieval performance
- Clustering quality

Human Evaluation:

- Topic interpretability
- Word intruder detection
- Topic utility for task

Important

Perplexity $\neq$ usefulness! Always check topic coherence and interpretability.

Applications of Classic Embeddings

1. Information Retrieval

- Semantic search
- Document similarity
- Query expansion

2. Document Organization

- Clustering
- Topic discovery
- Trend analysis

3. Text Mining

- Opinion mining
- Literature review
- Knowledge discovery

4. Preprocessing

- Feature reduction for ML
- Noise reduction
- Transfer learning

Real-world examples:

- Academic paper recommendations (LSA)
- News article categorization (LDA)
- Social media trend detection (BERTopic)

Can distributional semantics truly capture meaning?

Arguments For:

- Captures semantic similarity
- Works well in practice
- Aligns with linguistic theory
- Scalable to large corpora
- Unsupervised learning

Arguments Against:

- Text-only (no grounding)
- Missing embodied experience
- No common sense reasoning
- Cultural context limited
- Reflects biases in data

The Symbol Grounding Problem

"How can the semantic interpretation of a formal symbol system be made intrinsic to the system, rather than just parasitic on the meanings in our heads?" - Stevan Harnad (1990)

Reference: Harnad, S. (1990). "The symbol grounding problem"

1. Preprocessing Matters:

- Remove stopwords (but not always!)
- Lemmatization vs. stemming
- Handle punctuation carefully
- Consider bigrams/trigrams

2. Choosing Parameters:

- Start with 10-50 topics for LDA
- Use perplexity for model selection
- Validate with coherence scores
- Try multiple random seeds

3. Interpreting Results:

- Look at top 10-20 words per topic
- Examine representative documents
- Check for duplicate/junk topics
- Visualize with pyLDAvis

4. When to Use What:

- LSA: Fast exploration, search
- LDA: Interpretable topics
- BERTopic: Best quality, latest research

What we learned today:

1. **Distributional Hypothesis:** Words in similar contexts have similar meanings
2. **LSA (1990):**
 - SVD on term-document matrix
 - Linear dimensionality reduction
 - Fast but limited interpretability
3. **LDA (2003):**
 - Probabilistic topic modeling
 - Documents as mixtures of topics
 - Highly interpretable
4. **BERTopic (2022):**
 - Neural embeddings + clustering
 - State-of-the-art coherence
 - Automatic topic detection
5. **Evaluation:** Coherence $>$ perplexity
6. **Limitation:** Static representations, no grounding

Next: Neural word embeddings (Word2Vec, GloVe, FastText)!

Key References

Classic Papers:

- Firth, J.R. (1957). "A synopsis of linguistic theory 1930-1955"
- Deerwester et al. (1990). "Indexing by Latent Semantic Analysis"
- Blei et al. (2003). "Latent Dirichlet Allocation"
- Harnad, S. (1990). "The symbol grounding problem"

Modern Extensions:

- Boleda, G. (2020). "Distributional Semantics and Linguistic Theory"
- Grootendorst, M. (2022). "BERTopic: Neural topic modeling with a class-based TF-IDF procedure"
- Grand et al. (2022). "Semantic projection recovers rich human knowledge of multiple object features"

Tools & Libraries:

- Scikit-learn: `sklearn.decomposition.TruncatedSVD`, `LatentDirichletAllocation`
- Gensim: `gensim.models.LsiModel`, `LdaModel`
- BERTopic: <https://maartengr.github.io/BERTopic/>

Next Lecture:

Word Embeddings: Word2Vec, GloVe, FastText

From count-based to prediction-based methods!